Experiment No. 10

Title

Understanding INodes, I/O Redirection, Piping and Process Control Commands in Linux

Aim

The problem is to understand the concept of inodes, practice I/O redirection and piping, and use process control commands to manage running processes in a Linux environment.

Theory

Inodes

An inode is a data structure that stores metadata of a file such as:

- File size
- Ownership
- Permissions
- Timestamps
- Disk location

I/O Redirection

Redirection allows command outputs to be redirected to files instead of the terminal:

- > – Redirect output (overwrite)
- >> – Redirect output (append)
- < – Redirect input
- 2> – Redirect error output Pipes (|) allow the output of one command to be used as input to another command. Example: ls | grep txt

Pipes

Pipes (|) allow the output of one command to be used as input to another command.
Example: ls | grep txt

Process Control

Linux supports multitasking. Processes can be monitored and controlled using commands such as:

- ps
- top
- kill

- jobs
- bg, fg

Commands Used

Commands	Description
ls -i	Display inode number of files
stat	Display file inode details
>, >>	Output redirection
<	Input redirection
ps	Display running processes
top	Real-time process monitoring
kill	Terminate a process
jobs	List background jobs
bg	Resume job in background

Procedure / Steps

Step 1: Understanding Inodes

1. Create a test file: touch inode_test.txt
2. Display inode number: ls -i inode_test.txt
3. View inode details: stat inode_test.txt

```
ubuntu@seema:~$ touch inode_test.txt
ubuntu@seema:~$ ls i inode_test.txt
ls: cannot access 'i': No such file or directory
inode_test.txt
ubuntu@seema:~$ ls -i inode_test.txt
1984 inode_test.txt
ubuntu@seema:~$ stat inode_test.txt
  File: inode_test.txt
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: 0,29    Inode: 1984        Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2026-03-29 18:18:06.328368393 +0000
Modify: 2026-03-29 18:18:06.328368393 +0000
Change: 2026-03-29 18:18:06.328368393 +0000
 Birth: 2026-03-29 18:18:06.325368308 +0000
ubuntu@seema:~$
```

Step 2: Input / Output Redirection

4. Redirect output to a file: ls > file_list.txt
5. Append output: date >> file_list.txt
6. Redirect input: wc -l < file_list.txt

```
ubuntu@seema:~$ ls > file_list.txt
ubuntu@seema:~$ date >> file_list.txt
ubuntu@seema:~$ wc -l < file_list.txt
15
ubuntu@seema:~$
```

Step 3: Using Pipes

7. Combine commands using pipe: ls -l | grep ".txt"
8. Count number of text files: ls | grep ".txt" | wc -l

top

```
ubuntu@seema:~$ ls -l |grep ".txt"
drwxr-xr-x 2 ubuntu ubuntu  40 Mar 29 11:28 file1.txt
-rw-rw-r-- 1 ubuntu ubuntu 163 Mar 29 18:20 file_list.txt
-rw-rw-r-- 1 ubuntu ubuntu   0 Mar 29 18:18 inode_test.txt
ubuntu@seema:~$ ls | grep ".txt" | wc -l
3
ubuntu@seema:~$
```

Step 4: Process Control Commands

9. Display running processes: ps
10. Monitor processes in real-time: top
11. Run a command in background: sleep 60 &
12. List background jobs: jobs

13. Bring job to foreground: fg
14. Terminate a process: kill<PID>

```
ubuntu@seema:~$ ps
    PID TTY          TIME CMD
   6160 pts/0    00:00:00 bash
   6244 pts/0    00:00:00 ps
ubuntu@seema:~$ top
```

```
top - 18:29:09 up  1:10,  1 user,  load average: 0.00, 0.05, 0.14
Tasks: 315 total,   2 running, 313 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.9 us,  1.2 sy,  0.0 ni, 98.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1919.5 total,    212.8 free,   1305.4 used,    638.1 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    614.1 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2597 ubuntu    20   0  322936  75436  47432 S   2.3   3.8   1:06.03 Xorg
   2826 ubuntu    20   0 3807508 243368 104472 S   1.3  12.4   1:40.11 gnome-shell
   3413 ubuntu    20   0  398044   7968   6944 S   0.7   0.4   0:01.26 gvfs-afc-volume
   6148 ubuntu    20   0  850012  52968  42052 S   0.7   2.7   0:03.85 gnome-terminal-
   6281 ubuntu    20   0   23184   6024   3848 R   0.7   0.3   0:01.08 top
     11 root      20   0       0      0      0 I   0.3   0.0   0:04.84 kworker/0:1-event
   1524 root      20   0       0      0      0 S   0.3   0.0   0:00.47 l2arc_feed
   1626 systemd+  20   0   17560   7680   6912 S   0.3   0.4   0:03.91 systemd-oomd
   2932 ubuntu    20   0  236048   7660   6892 S   0.3   0.4   0:00.46 at-spi2-registr
   3119 ubuntu    20   0  398584  13448   7064 S   0.3   0.7   0:09.34 ibus-daemon
   3575 ubuntu    20   0  353840  27036  16240 S   0.3   1.4   0:08.92 ibus-extension-
   3784 ubuntu    20   0 1108936  92852  71732 S   0.3   4.7   0:01.59 xdg-desktop-por
      1 root      20   0   23380  14716   9724 S   0.0   0.7   0:14.99 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.11 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_re
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_fl
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.45 kworker/0:0H-kblo
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ip
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_perc
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kt
```

```
ubuntu@ubuntu:~$ sleep 60 &
[1] 5108
ubuntu@ubuntu:~$ jobs
[1]+  Running                 sleep 60 &
ubuntu@ubuntu:~$ fg
sleep 60
```

Outcomes

- ✓ Inode numbers and file metadata displayed correctly.
- ✓ Command output redirected and appended successfully.
- ✓ Pipes used to filter and process output.
- ✓ Processes monitored and controlled successfully.

Result

- ❖ Inode information, input/output redirection, piping, and process control commands were successfully executed. The student gained practical understanding of Linux file system internals and process management.